```
return threadId;
}
3D grid of 2D blocks
__device
int getGlobalIdx_3D_2D(){
int blockId = blockIdx.x + blockIdx.y * gridDim.x
 + gridDim.x * gridDim.y * blockIdx.z;
int threadId = blockId * (blockDim.x * blockDim.y)
 + (threadIdx.y * blockDim.x) + threadIdx.x;
return threadId;
}
3D grid of 3D blocks
__device
int getGlobalIdx_3D_3D(){
int blockId = blockIdx.x + blockIdx.y * gridDim.x
 + gridDim.x * gridDim.y * blockIdx.z;
int threadId = blockId * (blockDim.x * blockDim.y * blockDim.z)
 + (threadIdx.z * (blockDim.x * blockDim.y))
 + (threadIdx.y * blockDim.x) + threadIdx.x;
return threadId;
}
```

Solved Exercise: Program to add two numbers.

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"

__global__ void add(int *a, int *b, int *c)
        {
                *c = *a + *b;
        }

int main(void) {
                int a, b, c;                    // host copies of variables a, b & c
                int *d_a, *d_b, *d_c;           // device copies of variables a, b & c
                int size = sizeof(int);

                // Allocate space for device copies of a, b, c
                cudaMalloc((void **)&d_a, size);
                cudaMalloc((void **)&d_b, size);
                cudaMalloc((void **)&d_c, size);
                // Setup input values
                a = 3;
                b = 5;
// Copy inputs to device
                cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
                cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);
                // Launch add() kernel on GPU
                add<<<1,1>>>(d_a, d_b, d_c);
                // Copy result back to host
                cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);
                printf("Result : %d",c);
                // Cleanup
                cudaFree(d_a);
                cudaFree(d_b);
                cudaFree(d_c);
                return 0;
        }
```

Explanation:

add is the function which runs on device.

```
cudaMalloc ((void **)&d_a, size);
```

cudaMalloc will allocate memory of size bytes given as second argument to variable passed as first argument.

```
cudaMemcpy (Destination,Source,Size,Direction);

cudaMemcpy (d_a, &a, size, cudaMemcpyHostToDevice);

cudaMemcpy (&c, d_c, size, cudaMemcpyDeviceToHost);
```

cudaMemcpy copies the variables from host to device or device to host based on the direction which is either cudaMemcpyHostToDevice or cudaMemcpyDeviceToHost. Value of size bytes long is copied from source to destination.

cudaFree frees the memory allocated by cudaMalloc.

The add function is called like this add<<<1,1>>>(d_a,d_b,d_c). The add is followed by three angular brackets then the number of blocks, threads per block then corresponding closing angular brackets then how many arguments the function add takes is enclosed within parenthesis. If you want to add N elements you can achieve it in two ways either having N blocks or having N threads.

That is pass an array with following function calls

add<<< N,1>>> (d_a,d_b,d_c) or add<<< 1, N>>> (d_a,d_b,d_c)

Few Mathematical functions in CUDA:

Major Single-Precision floating point functions: Single precision functions work on float value(32 bit). A float value is stored in IEEE 754 format.

Function	**Description**
sqrtf(x)	Square root function
expf(x)	Exponentiation function. Base = e
exp2f(x)	Exponentiation function. Base = 2
exp10f(x)	Exponentiation function. Base = 10
logf(x)	Logarithmic function. Base=e
log2f(x)	Logarithmic function. Base=2
log10f(x)	Logarithmic function. Base=10
sinf(x)	sine function

cosf(x)	cos function
tanf(x)	tan function
powf(x,y)	power function
truncf(x)	truncation function
roundf(x)	round function
ceilf(x)	ceil function
floorf(x)	floor function

Major Double-Precision floating point functions: Double precision functions work on double value(64 bit). A double value is stored in IEEE 754 format.

Function	**Description**
sqrt(x)	Square root function
exp(x)	Exponentiation function. Base = e
exp2(x)	Exponentiation function. Base = 2
exp10(x)	Exponentiation function. Base = 10
log(x)	Logarithmic function. Base=e
log2(x)	Logarithmic function. Base=2
log10(x)	Logarithmic function. Base=10
sin(x)	sine function
cos(x)	cos function
tan(x)	tan function
pow(x,y)	power function
trunc(x)	truncation function
round(x)	round function